Mathematical Methods in Uncertainty Quantification (MMUQ) – L05

Faizan Anwar (faizan.anwar@tum.de)

Technical University of Munich

School of Engineering and Design

Chair of Hydrology and River Basin Management (Prof. Disse)

MMUQ L05 – Cython

- Introduction
- Uses
- Requirements
- Coding properly
- Cythonization steps

Cython

- A superset of Python
- A language that adds to Python
- Compiled is much faster than interpreted
- Additional syntax to make Python compilable
- Achieves by converting to C/C++ code
- Regular Python code can also be directly converted to C/C++

Uses

- Mathematical Operations
- Array-based computation
- Parallelization
  - Using OpenMP
- Datatype enforcing

Requirements

- pip install cython
- Two modules installed
  - cython
  - pyximport
- A C/C++ compiler
  - The same one used to compile the Python interpreter
- .pxd file
  - Contains function declarations
- .pyx file
  - Contains Cythonized code
- .pyxbld
  - Contains info on imported libraries, and compiler options

Cython Coding

Inside a .py file

```
def add(a, b):
    return a + b
```

Inside a .pyx file

```
cpdef double add(double a, double b):
    return a + b
```

or (better)

```
def add(a, b):
    return _add(a, b)

cdef double _add(double a, double b) nogil:
    return a + b
```

Cythonization Steps

- Write the required function in Python
  - Debug
  - Verify everything is as intended
- Read the Cython documentation
- Create a copy inside a .pyx file
  - with `nogil`
  - Turnoff index check
- Declare all variables inside
  - Datatypes (`int`, `float`, etc.)
  - using `cdef`
- Memoryviews for typed arrays
  - `x` → `np.float32[:, ::1] x`

- Create .pxd with declaration
  - Just the function signature
- Create the .pyxbld
  - With compiler options
  - Some option for numpy
- Only cythonize the computationally demanding part!

HBV001a example…

That was it for today, thanks!